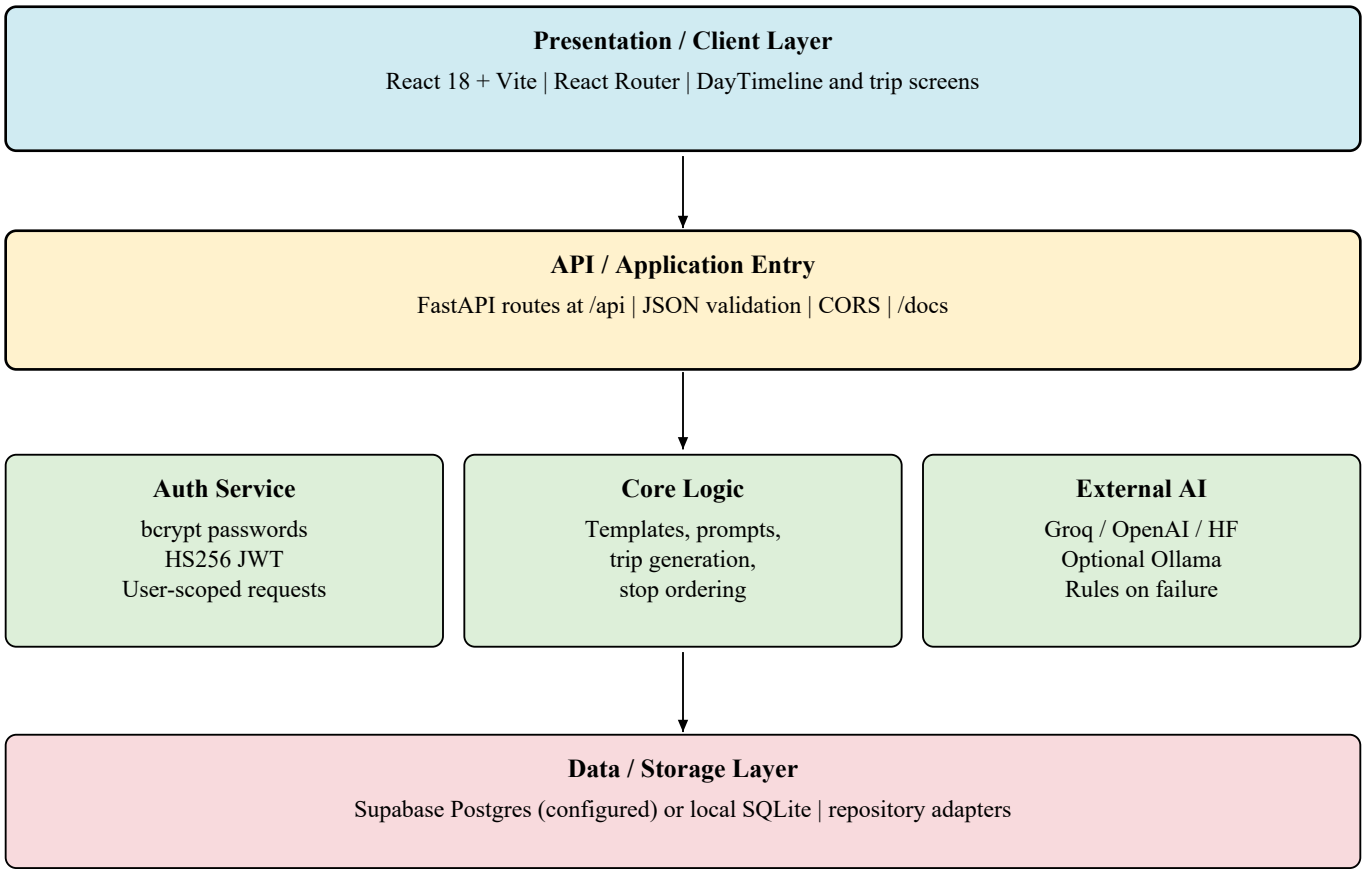
Solution Architecture

Date	23 October 2026
Team ID	0827CY231053
Project Name	Waypoint - AI Travel Itinerary Planner
Maximum Marks	5 Marks

Solution Architecture Diagram

A React single-page application calls one modular FastAPI backend. The application uses repository adapters for Supabase or SQLite; the internal services are modules within this backend.



The entry box represents the FastAPI API service. A separate gateway or load balancer is not configured in the source project.

Solution Architecture (continued)

Component Description Table

Component Name	Description / Role in Architecture	Technologies Used
Presentation Layer	Collect inputs, display curated trips and daily timelines, manage saved trips and show public plans.	React 18, Vite, React Router, Framer Motion, Lucide, CSS
API / Application Entry	Expose /api routes; validate inputs; configure CORS and error responses; publish /docs.	FastAPI, Pydantic, Uvicorn/Gunicorn
Auth Service	Hash passwords, verify credentials, issue signed seven-day JWTs and require tokens on private routes.	bcrypt; python-jose; HTTPBearer
Core Logic	Prepare prompts, generate day plans, rank destinations, order stops and format exports.	Python service modules; curated catalog; trip routers
AI Integrations	Select a configured model provider; receive JSON output; use local rules on failure.	Groq, OpenAI, Hugging Face or optional Ollama; httpx
Database	Persist users, preferences, trips, days, activities and recommendation logs.	Supabase Postgres or SQLite; repository adapters

Configuration and Availability

Database mode is selected at startup from Supabase configuration. SQLite is a local mode, not a live failover mirror of production data. AI generation can continue using local rules when the selected model is unavailable.

Project sources: backend main.py, store.py, config.py, security.py, services/ and database schemas; frontend App.jsx.